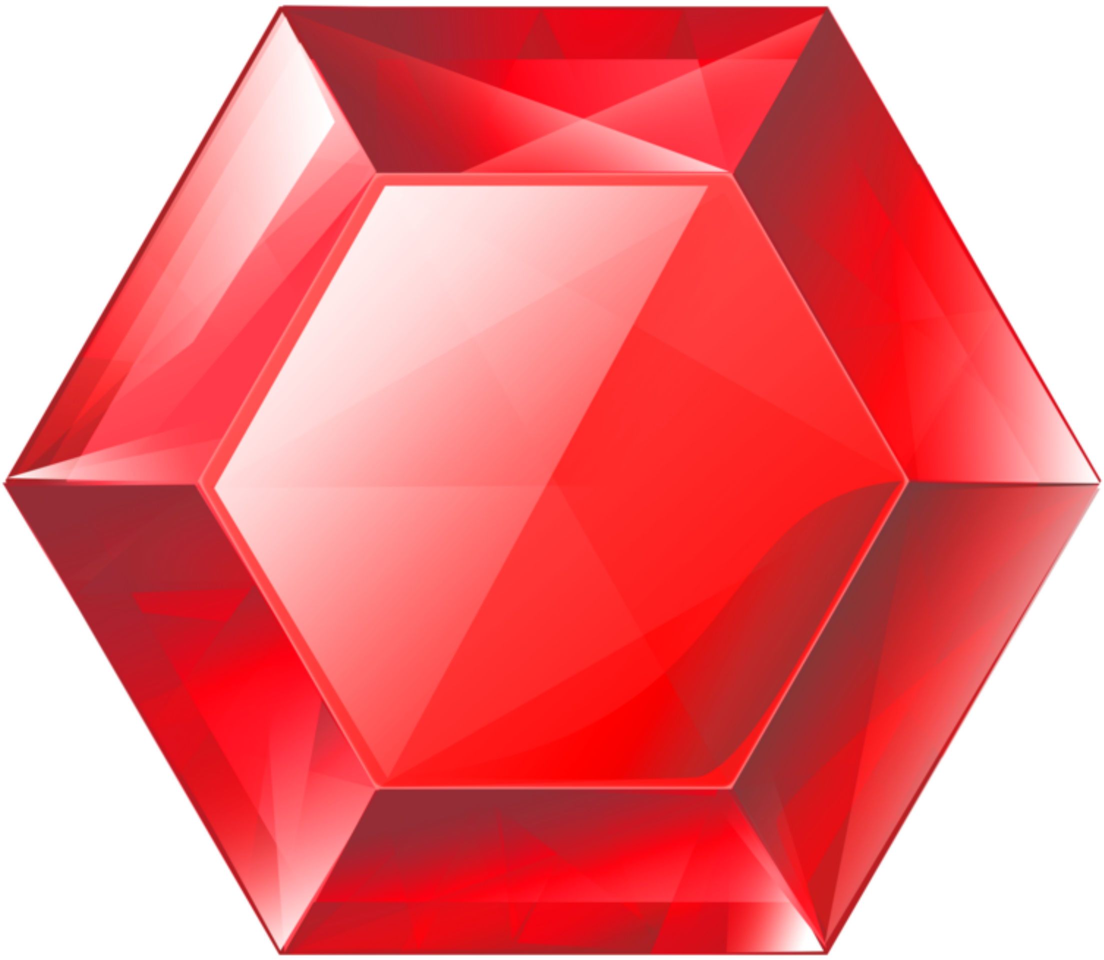
gemstone-py vs gemstone-rs

Install paths, use cases, maturity, and feature matrix



Generated from the Markdown sources in gemstone-rs/docs.

Contents

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	`python -m pip install gemstone-py`	`cargo add gemstone-rs`
Native acceleration	`python -m pip install "gemstone-py[fast]"`	Built into the Rust GCI path once `libgcirpc` is available
Examples from source	`python -m pip install -e ".[examples]"`	`cargo run -p gemstone-rs --example quickstart`
Example discovery/run	`gemstone-examples hello`, `list`, `plan3-map`, and launch commands	`gemstone-rs hello`, `compare gemstone-py`, `examples list`, `map`, `show`, source-checkout `run`, and installed `scaffold` templates
CLI tooling	Python modules and examples	`cargo install gemstone-rs-cli`
Local explorer	`python-gemstone-database-explorer`	`cargo install gemstone-rs-explorer`
VS Code	`gemstone-py Workbench`	`gemstone-rs Workbench`

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
Example-driven onboarding	gemstone-py today, gemstone-rs catching up	gemstone-py has broader runnable examples; gemstone-rs now has no-live hello, a direct comparison command, an installed CLI example index, feature map, source-checkout runner, and standalone project scaffolds.
VS Code database/ codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added
Native bridge	PyO3 extension path	Rust GCI crate and safe API
Sync API	Mature	Initial safe API
Async API		Dedicated-thread `SessionWorker` and bounded `SessionWorkerPool`; async facade still planned

Capability	gemstone-py	gemstone-rs
	Mature enough for examples and tests	
Web frameworks	FastAPI, Litestar, Django examples	Shared `gemstone_rs::web` health helpers, standard-library HTTP, `SessionWorkerPool`, packaged `gemstone-rs-axum` / `gemstone-rs-actix` adapters, and checked Axum/Actix examples
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/generate, live discovery, typed argument conversion, typed return helpers, and profile scaffolds
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/source
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, example index, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported
`3 + 4 == 7` smoke	Supported	Supported
Global put/get	Supported	Supported
String round-trip	Supported	Supported

Feature	gemstone-py status	gemstone-rs status
`perform` and `perform_oop`	Supported	Supported
Commit/abort	Supported	Supported
Dedicated session worker	Supported through Python web integration patterns	Supported through `SessionWorker` and `SessionWorkerPool`
OOP handles/export set	Supported	Supported
Browser dictionaries/ classes/protocols/ methods/source	Supported	Supported through `Browser`, CLI, and explorer
Generated wrappers	Supported	Supported for selected classes/methods
Generated typed arguments	Python naturally passes Python values	Supported through `args=name:Type` for `SmallInt`, `String`, `Symbol`, `Bool`, and explicit `Oop`
Codegen diff/check/ generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API and installed scaffolds
FastAPI demo	Supported	Not applicable
Litestar demo	Supported	Not applicable
Rust HTTP service demo	Not applicable	Supported through `cargo run -p gemstone-rs --example http_service -- --port 3000`
Axum/Actix demo	Not applicable	Axum and Actix services supported through `gemstone-rs-axum`, `gemstone-rs-actix`, `examples/axum-service`, and `examples/actix-service`
Installed examples index	`gemstone-examples hello`, `list`, `plan3-map`	`gemstone-rs hello`, `compare gemstone-py`, `examples list`, `map`, `show`, `run --dry-run`, `scaffold`, plus JSON for tooling
Database explorer	Mature Python app	Rust explorer has browser UI and local API; still less polished

Feature	gemstone-py status	gemstone-rs status
VS Code sidebar workflow	More complete	Rust workbench has sidebar commands and embedded explorer webview

Actionable Gap Report

The CLI can print the remaining gemstone-py parity gaps directly:

```
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --gaps --json
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --scorecard --json
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --parity --json
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --next --json
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --totals --json
gemstone-rs compare gemstone-py --batches
gemstone-rs compare gemstone-py --batches --json
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
```

`compare all --batches` combines this Rust/Python track with the TypeScript/Python track and reports **12 batches**, roughly **86-151 hours** total.

The report is intentionally action-oriented. Each row names the gemstone-py strength, the gemstone-rs gap, the next implementation step, and the command or test that should verify it.

Use `--scorecard` when you want the short decision answer instead of the full gap report. It summarizes where gemstone-py is stronger today, where gemstone-rs is stronger today, when to choose each project, how many batches remain, and the next recommended batch.

Use `--parity` when you want a measured maturity view. It scores each area out of five and shows the current leader, status, and next action. The current Rust/Python parity score is **gemstone-py 30/35** and **gemstone-rs 25/35**: Rust is at parity or ahead for core sessions, codegen/mapping, and the shared native-core direction; Python remains ahead for web frameworks, async/lifetime coverage, explorer polish, and release lane depth.

Priority	Area	What gemstone-py has today	gemstone-rs next action
P1			

Priority	Area	What gemstone-py has today	gemstone-rs next action
	Web framework adapters	FastAPI, Litestar, and Django examples are first-class.	Add request tracing, middleware examples, and stricter live route smoke coverage around the packaged `gemstone-rs-axum` and `gemstone-rs-actix` crates.
P1	Explorer product polish	The Python database explorer is the richer class browser and product reference.	Make the embedded Rust explorer webview the primary IDE surface for browsing, codegen, diff, and BridgeRoot inspection.
P1	Installed example experience	`gemstone-examples` launches installed examples without a source checkout.	Expand `gemstone-rs examples scaffold` to explorer-integrated projects and richer generated wrapper profile variants.
P2	Async facade	gemstone-py has async examples and FastAPI integration.	Add an async facade over `SessionWorkerPool` after GCI thread behavior is proven with live tests.
P2	Shared native core	gemstone-py already exposes Python packaging and optional native acceleration.	Make `gemstone-py-native` a thin PyO3 adapter over `gemstone-gci` and `gemstone-rs`.
P2	Release lane depth	gemstone-py has mature PyPI/TestPyPI/native wheel/VSIX release lanes.	Exercise the full crates.io, Marketplace, GitHub Release, PDF, and checksum workflow regularly.

Use `--next` when you only want the first recommended implementation step:

```
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --next --json
```

Remaining Work Batches

The batch plan is also available from the CLI:

```
gemstone-rs compare gemstone-py --batches
gemstone-rs compare gemstone-py --batches --json
gemstone-rs compare gemstone-py --totals
gemstone-rs compare all --totals
```

Batch	Work	Estimate
1	Explorer and VS Code webview polish	10-18 hours
2	Object mapping maturity	8-14 hours
3	Codegen live discovery and generated tests	8-14 hours
4	Async facade and web middleware	6-12 hours
5	Shared core with `gemstone-py-native`	8-14 hours
6	Release and live CI hardening	4-7 hours

Total: roughly **44-79 hours** to bring `gemstone-rs` materially closer to `gemstone-py` across product polish, generated-code confidence, async/web ergonomics, shared native-core integration, and release depth.

Current Gap Analysis

`gemstone-py` remains better when the user wants a batteries-included Python product surface: package extras, web adapters, async examples, a mature database explorer, and an examples launcher aimed at Python users and installed environments.

The most useful gemstone-rs catch-up work is to keep copying the product shape, not the implementation language:

- Installed discoverability: `gemstone-rs examples list` now mirrors the gemstone-py example map and gives VS Code structured JSON.
- No-live sanity check: `gemstone-rs hello` mirrors `gemstone-examples hello` so users can verify the CLI before configuring GemStone.
- Direct comparison: `gemstone-rs compare gemstone-py` turns this guide into a compact CLI summary with JSON output for tooling.
- Feature stream map: `gemstone-rs examples map` mirrors `gemstone-examples plan3-map`, but frames each stream in Rust crates, examples, docs, and gemstone-py reference points.
- Source-checkout launching: `gemstone-rs examples run <name>` now executes the matching Cargo example, with `--dry-run` for CI and documentation checks.

- Installed project scaffolding: `gemstone-rs examples scaffold quickstart`,

  `browser`, `bridge_root_mapping`, `derive_mapping`, `codegen_preview`, `codegen_workflow`, `codegen_discover`, `codegen_discover_mapping`, `profile_codegen_workflow`, `generated_wrapper_app`, `generated_mapping_app`, `http_service`, `axum_service`, and `actix_service` create standalone Cargo projects from the installed CLI, reducing the need for a source checkout. `profile_codegen_workflow` includes codegen config and project profile files, not only Rust source.
- Rust web service shape: `http_service` gives a real HTTP example with `/`,

  `/health/local`, and `/health/gemstone` without adding framework dependencies. `gemstone-rs-axum` and `gemstone-rs-actix` now package the same route contract for framework users, and `examples/axum-service` plus `examples/actix-service` are checked crates using those adapters. They can start before credentials are configured and report `/health/gemstone` as a `503` JSON error until the pool is available. They also emit diagnostic adapter and route headers for route smoke tests and proxy logs. The installed CLI can scaffold `session_worker_pool`, `axum_service`, and `actix_service`. Richer framework middleware and an async facade remain future work.
- Editor workflow: `GemStone RS: Show Example Commands` exposes the same map in

  the Rust workbench and can run selected Cargo examples in a terminal.
- Remaining gap: gemstone-rs still needs installed templates for

  explorer-integrated workflows and richer generated wrapper profile variants, plus framework middleware, an async facade, and richer explorer screens before its onboarding feels as complete as gemstone-py.

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` can become a thin PyO3 layer over the Rust core.
- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on

  common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.